CONFIDENTIAL

PROPEL · ENGINEERING

full-stack ai engineer.

take-home assignment. about five hours of work, seventy-two hours to send it back. we'd rather see a small thing that runs than a large thing that doesn't.

ROLE

full-stack ai engineer

PRODUCT

Dhaba · practice product

TIME

~5 hours · 72 hours to submit

confidential · prepared for one named candidate · do not forward, publish or reuse

PROPEL · TAKE-HOME

before you start

this is close to something we actually run. support volume across our apps is handled by agents with humans on top, and the hard part was never the prompt — it was everything around it: schemas that hold, money decisions that can't be talked into, and logs you can read at 3am.

about **five hours of work**, **72 hours** from when you receive it. use any language and stack you'd defend. use whatever model you have access to — if you have no paid api key, say so and use a free tier or a local model; it will not count against you.

the product you're working with

Dhaba is a practice product. we invented it for this assignment — it is not one of ours and there is nothing to look up.

a food app with the money model most of our portfolio uses: a ₹1 trial purchase, then 24 hours later the real plan activates on its own — ₹249 a month or ₹1,499 a year. purchases are in one of four states: initiated, failed, successful, renewal. users write in through an in-product help form.

> ONE THING WORTH KNOWING
>
> if something here isn't clear, ask. asking a good question is a point in your favour, not a penalty. a decision you made and wrote down in the readme is also fine. guessing silently is the thing we're watching for.

the file

`dhaba_tickets.json` is attached to the email you got this on: twelve real-shaped tickets, each with what billing knew about that user at the moment the ticket landed. english, hinglish, one-word, and angry are all represented, because that is what the inbox looks like.

task 1 — the service · 180 MIN

one http endpoint, `POST /triage`. it takes a ticket exactly as it appears in the file and returns json a support tool could act on:

```
category: one of a small enum you define
severity: your scale, documented
refund: whether to refund, how much, and why
reply_draft: what we'd send the user, in their language
needs_human: true when a person must look
confidence: and what you do when it's low
```

what we are actually reading in this task:

- **the output holds its shape.** the model will eventually return prose, truncated json, or an invented category. decide what happens then — validate, retry, degrade — and make that path real, not a comment.
- **money has a separate gate.** a refund decision should not be a field the model can freely write. where you draw that line is most of our read on you.
- **it runs twice safely.** the same ticket id arriving again must not refund twice.
- **logs are clean.** nothing in these tickets that identifies a person or a company should end up in a log line.
- **we can run it without an api key.** a replay or fixture mode, so your evaluator isn't burning your tokens.

task 2 — make it not lie · 45 MIN

run all twelve tickets through it and put the results in a table in your readme: what it decided, and whether you agree with it.

- a few of these tickets are genuinely ambiguous. say which, and say what your system does when it can't tell — that answer matters more than the accuracy number.
- **two of the tickets contain text addressed to the assistant rather than to us.** find them. say what your service does with them and show the output that proves it. if it did what the text asked, say that too — we would rather read an honest failure than a quiet one.
- one number you'd put on a dashboard to know this thing is still working next month.

task 3 — your track · 45 MIN

pick the one that matches the craft you'd bring. do one, not both.

backend. this endpoint is about to get fifty tickets a second during an outage, each one wanting a model call.

- what you change so it degrades instead of falling over, and what the user gets while it's degraded.
- what you store and where — sketch the tables or documents.
- one thing you'd add now that makes debugging this at 3am possible.

mobile. build the screen the support agent works from, android and Compose.

- a queue, and one ticket open: the user's history, the triage result, the drafted reply.
- approve, edit, escalate — with the edit going back as the thing we actually send.
- point it at your service, or a stub if you ran out of time. state which.

task 4 — how you used ai · 15 MIN

- which agents you used and roughly what share of the code they wrote.
- your single best prompt, pasted exactly as you sent it.
- **one thing an agent got wrong here** — a plausible-looking function that was subtly incorrect, a test that asserted nothing, a library it invented — how you caught it and what you changed. an answer saying it was right about everything reads as either no ai or no checking.
- one part you wrote by hand because directing an agent was slower, and why.

---

how to send it back

reply to the same email thread you got this on with a github link — public, or private with `propel-hiring` invited — containing the service, your readme with tasks 2, 3 and 4 in it, and exact steps to run it from a clean machine. commit as you go; we read the history.

if we like it, the next step is a **45-minute live pairing session** on this same code, with Sourav. we'll change the requirements while you're in it. that's the actual job.

> **CONFIDENTIAL**
>
> this document and the ticket data attached to it are Propel internal material, shared with you alone for this assessment. please don't forward it, post it, or use it in a portfolio.
>
> the code you write stays yours. we won't ship it or use it in any product.